

DSC1105_MONFERO_FA3

John Benedict A. Monfero

2025-02-24

```
library(tidyverse)
```

```
## — Attaching core tidyverse packages — tidyverse 2.0.0 —
## ✓ dplyr      1.1.4    ✓ readr      2.1.5
## ✓ forcats    1.0.0    ✓ stringr    1.5.1
## ✓ ggplot2    3.5.1    ✓ tibble     3.2.1
## ✓ lubridate  1.9.4    ✓ tidyr      1.3.1
## ✓ purrr      1.0.2
## — Conflicts — tidyverse_conflicts() —
## ✗ dplyr::filter() masks stats::filter()
## ✗ dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

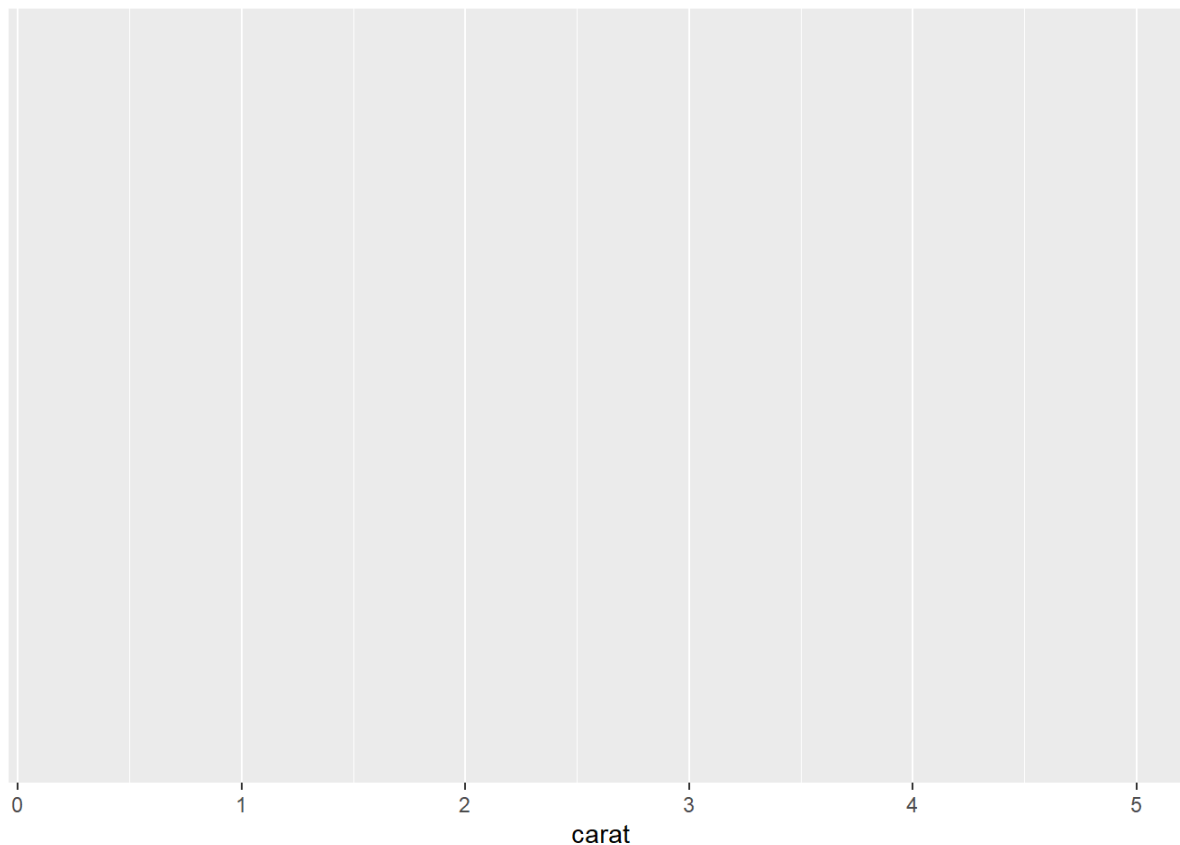
```
diamonds
```

```
## # A tibble: 53,940 × 10
##   carat cut      color clarity depth table price      x      y      z
##   <dbl> <ord>    <ord> <ord>    <dbl> <dbl> <int> <dbl> <dbl> <dbl>
## 1  0.23 Ideal     E     SI2     61.5    55   326  3.95  3.98  2.43
## 2  0.21 Premium  E     SI1     59.8    61   326  3.89  3.84  2.31
## 3  0.23 Good     E     VS1     56.9    65   327  4.05  4.07  2.31
## 4  0.29 Premium  I     VS2     62.4    58   334  4.2   4.23  2.63
## 5  0.31 Good     J     SI2     63.3    58   335  4.34  4.35  2.75
## 6  0.24 Very Good J     VVS2    62.8    57   336  3.94  3.96  2.48
## 7  0.24 Very Good I     VVS1    62.3    57   336  3.95  3.98  2.47
## 8  0.26 Very Good H     SI1     61.9    55   337  4.07  4.11  2.53
## 9  0.22 Fair     E     VS2     65.1    61   337  3.87  3.78  2.49
## 10 0.23 Very Good H     VS1     59.4    61   338  4     4.05  2.39
## # i 53,930 more rows
```

Create a histogram on the diamonds dataset

```
# Let `p` be the container of the histogram visualization below
p <- ggplot(data = diamonds, aes(x = carat))
```

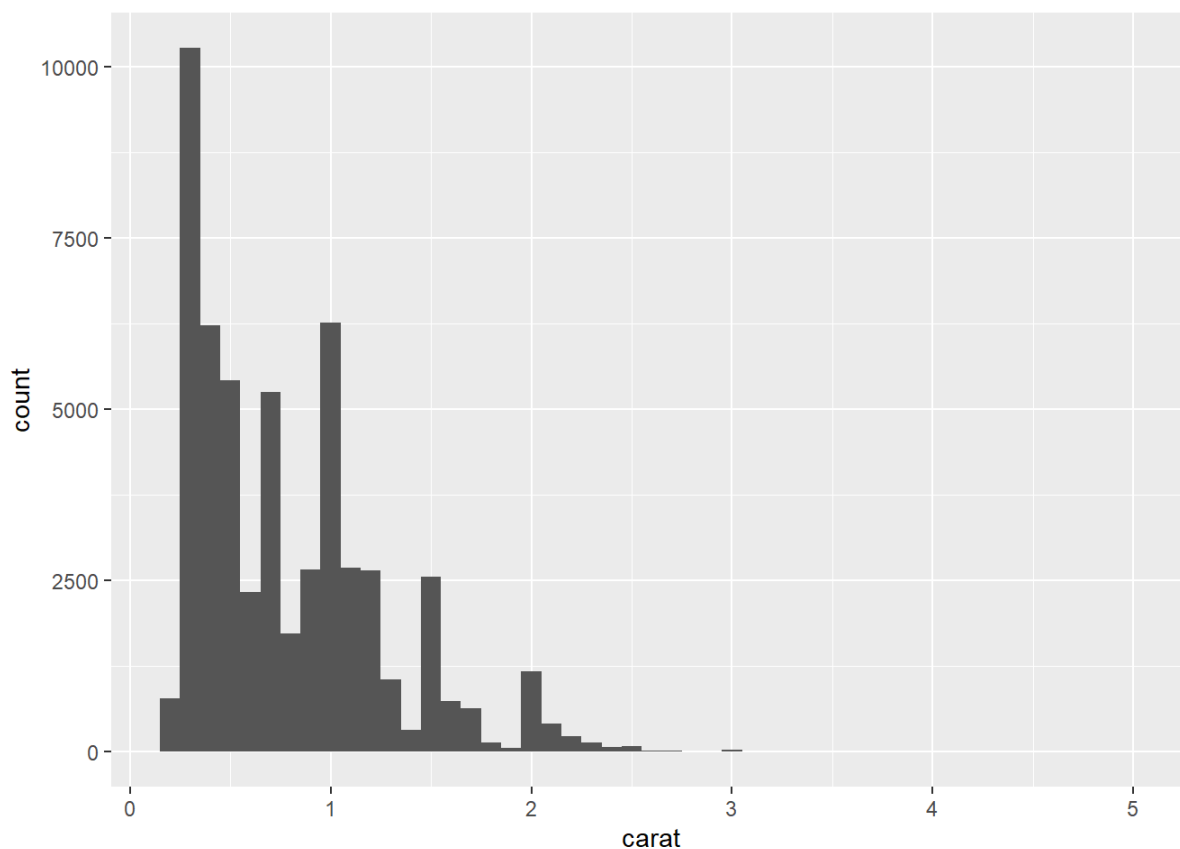
```
p # The figure below only shows a blank canvas but the x-axis is all about carat of the diamond dataset
```



Re-write this using the layer function like we did in class.

```
# Hint: if you don't know what the default values for some of the aspects of the plot, examine p$layers.
p <- p + layer(
  geom = "bar",
  stat = "bin",
  position = "stack",
  mapping = aes(x = carat),
  params = list(binwidth = 0.1)
)

# Recall p
p # p <- p$layer would give us the proper histogram visualization that we had like in FA1 but with the code:
```



```
# ggplot(data = diamonds) +
# geom_histogram(mapping = aes(x = carat))
```

Remember that a histogram is a plot with `stat_bin` and `geom_bar`.

Modify your histogram code so that it uses a different `geom`, for example `geom_line` or `geom_point`.

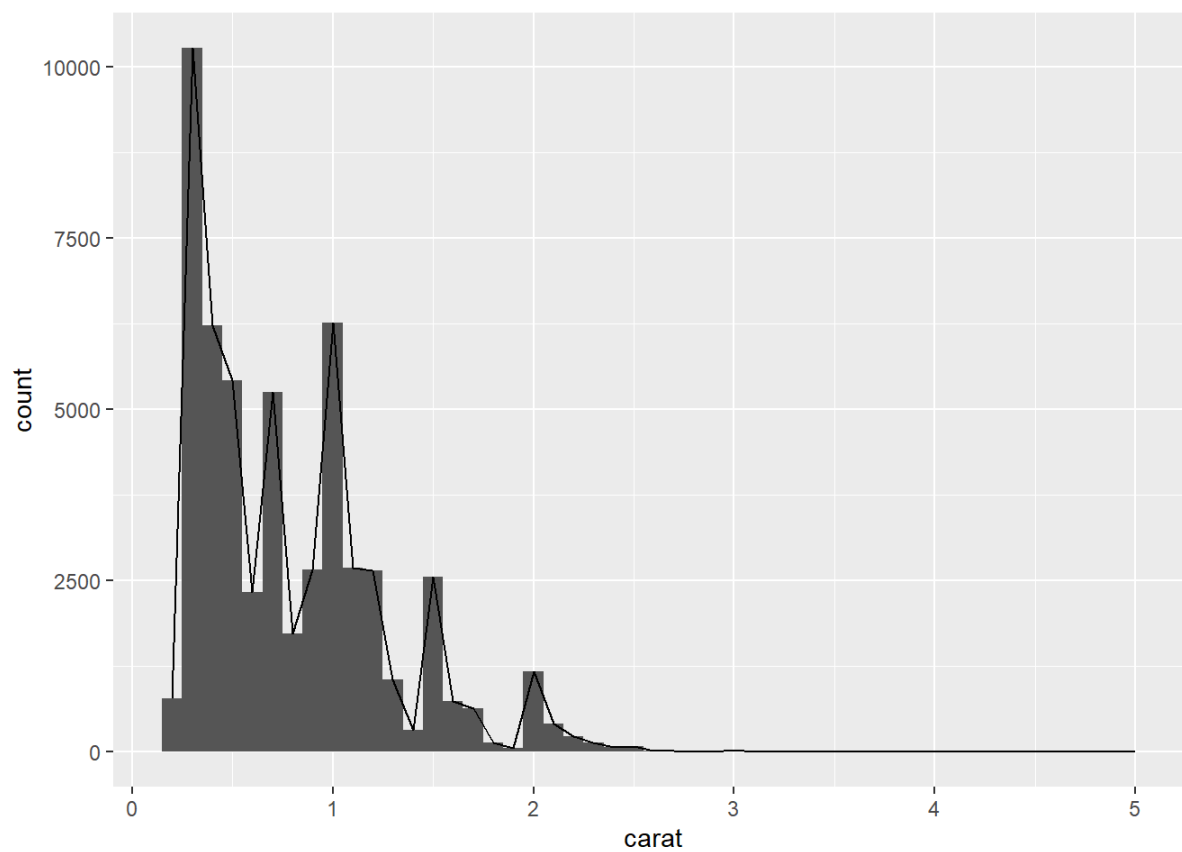
```
# This should be simple once you have the p$layer specification of a histogram.

# Let `q` and `r` be the modified histogram codes from p$layer specification with as `geom_line` and `geom_point` respectively.
```

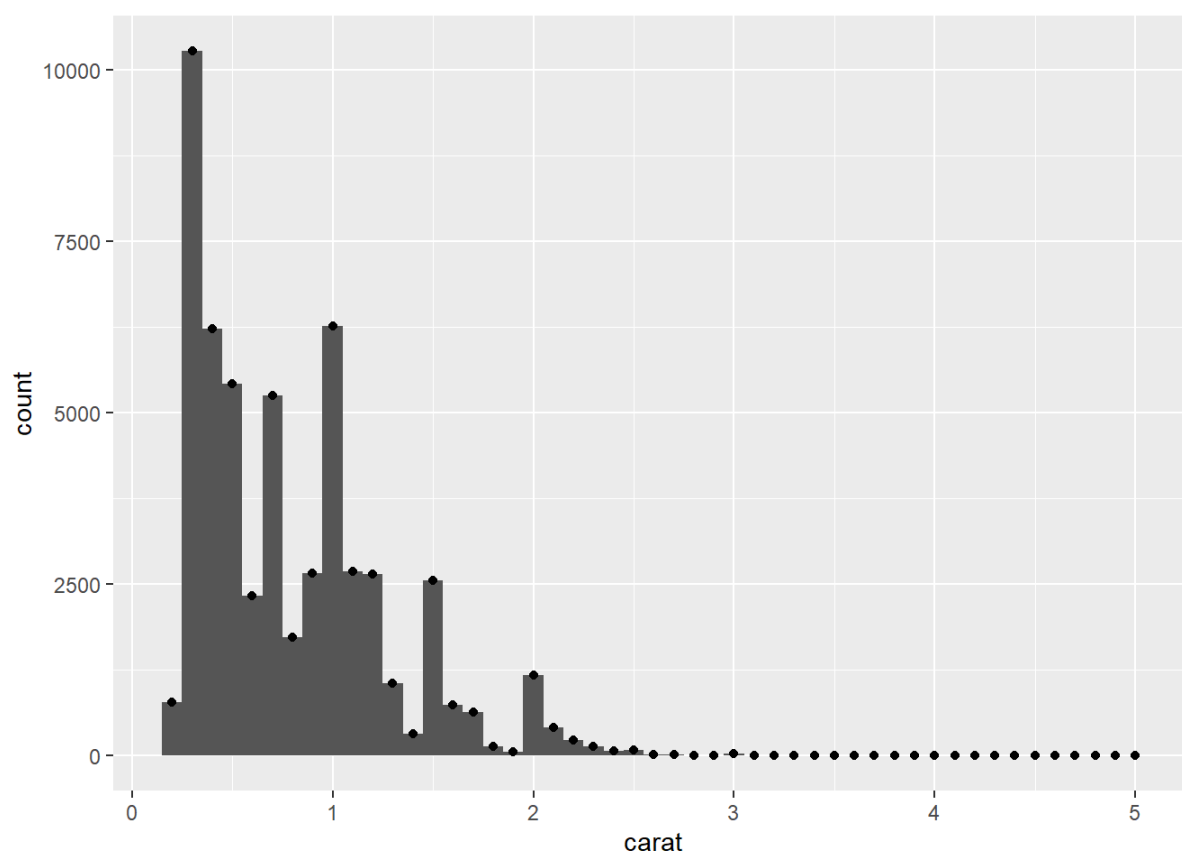
```
q <- p + layer(
  geom = "line",
  stat = "bin",
  position = "stack",
  mapping = aes(x = carat),
  params = list(binwidth = 0.1)
)
```

```
r <- p + layer(
  geom = "point",
  stat = "bin",
  position = "stack",
  mapping = aes(x = carat),
  params = list(binwidth = 0.1)
)
```

```
# Recall q
q
```



Recall r
r



In your histogram in container `p` (the one plotted with bars that you

created first):

```
# This is how object `p` visualizes its histogram given the `mapping`, `geom_bar`, `stat_bin`, `position`  
`, and other parameters necessary  
p$layers
```

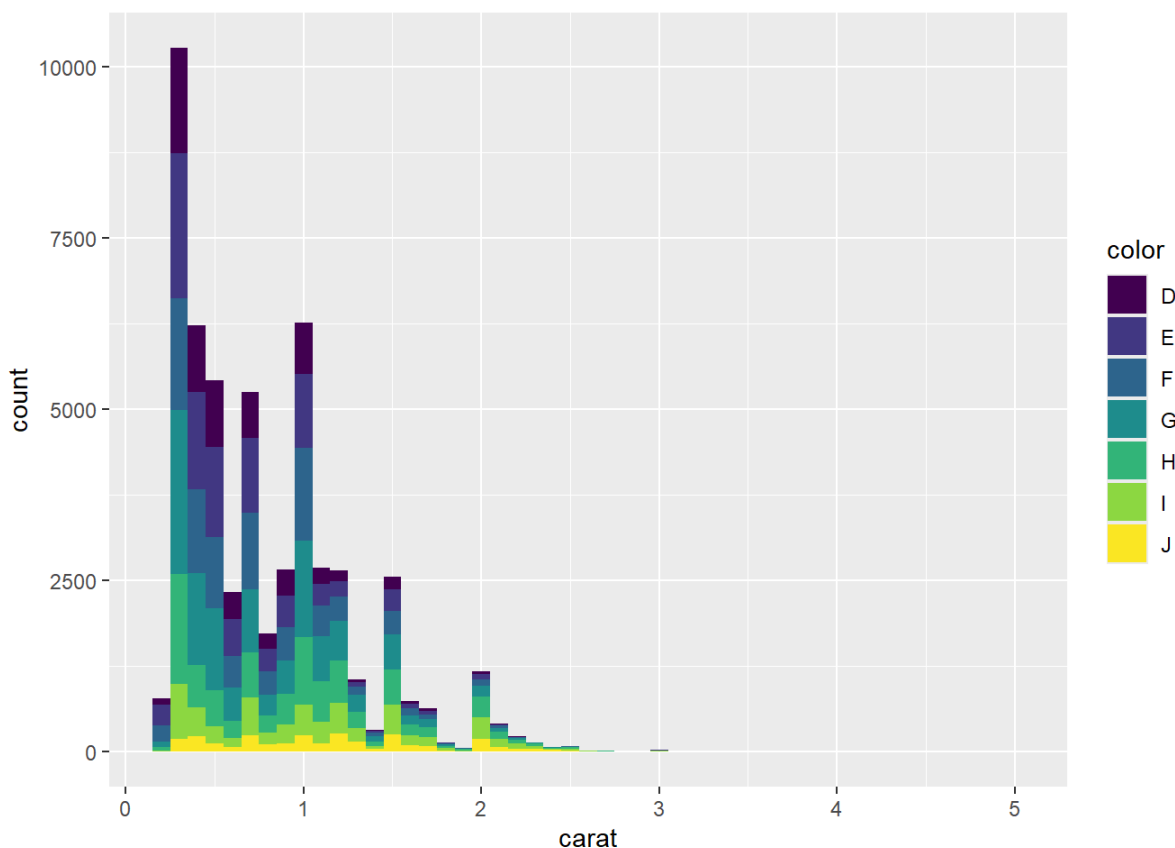
```
## [[1]]  
## mapping: x = ~carat  
## geom_bar: na.rm = FALSE  
## stat_bin: binwidth = 0.1, na.rm = FALSE  
## position_stack
```

Add an aesthetic mapping from one of the factor variables (maybe color or clarity) to the fill or color aesthetic.

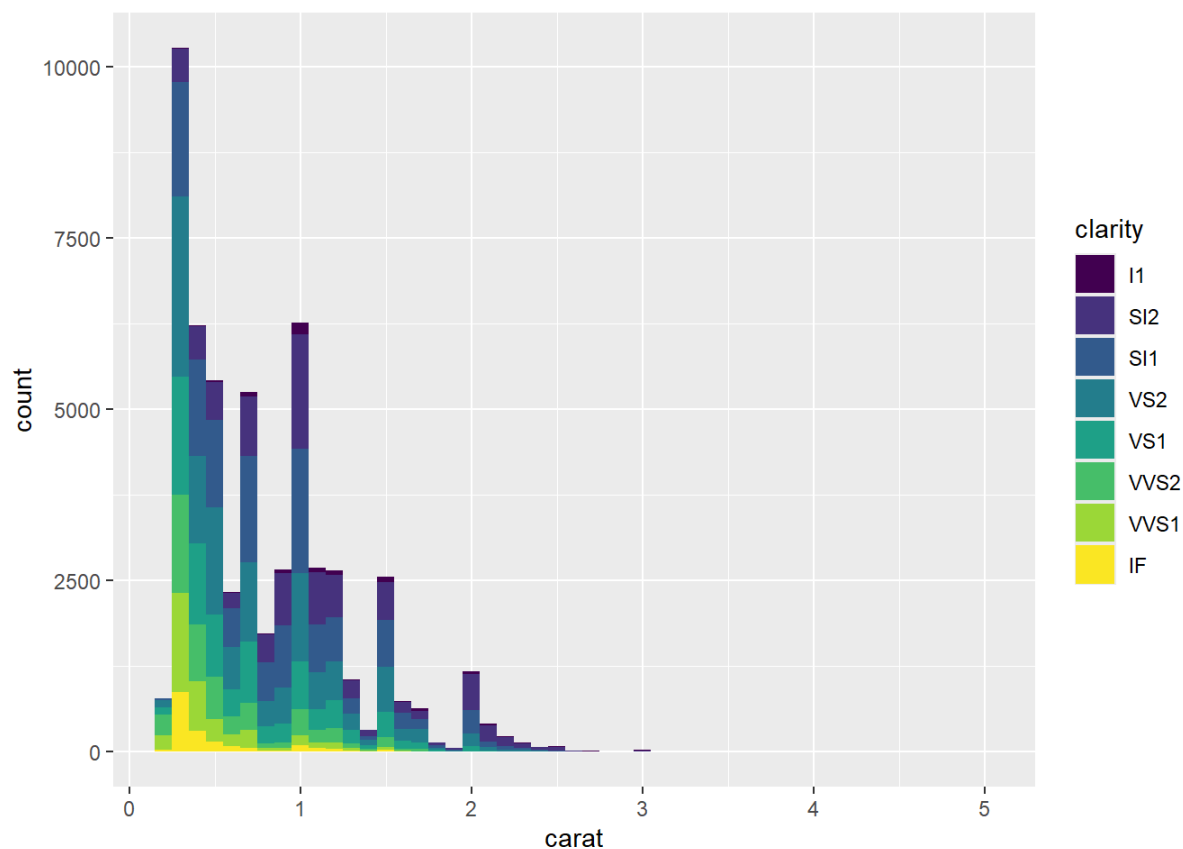
```
# Let `s` and `t` be the modified histogram codes from p$layer but modified according to their aesthetic  
s
```

```
s <- ggplot(data = diamonds, aes(x = carat, fill = color)) +  
  p$layers  
  
t <- ggplot(data = diamonds, aes(x = carat, fill = clarity)) +  
  p$layers
```

```
# Recall s  
s
```



```
# Recall t  
t
```



What is the default position adjustment for a histogram contained in `s` and `t`?

```
s$layers
```

```
## [[1]]
## mapping: x = ~carat
## geom_bar: na.rm = FALSE
## stat_bin: binwidth = 0.1, na.rm = FALSE
## position_stack
```

```
print("As shown above, the default position adjustment for a histogram contained in `s` is `position_stack`")
```

```
## [1] "As shown above, the default position adjustment for a histogram contained in `s` is `position_stack`"
```

```
t$layers
```

```
## [[1]]
## mapping: x = ~carat
## geom_bar: na.rm = FALSE
## stat_bin: binwidth = 0.1, na.rm = FALSE
## position_stack
```

```
print("As shown above, the default position adjustment for a histogram contained in `t` is `position_stack`")
```

```
## [1] "As shown above, the default position adjustment for a histogram contained in `t` is `position_stack`"
```

Try changing the position adjustment in the histogram you created to something different.

```
# If we have complicated layers already, it is also complicated to replicate everything and change only one specific argument to perform the current task!
```

```
# We can change some of the properties of s$layer and t$layers without calling all parameters in layer() again:
```

```
# Hint: try position = dodge
```

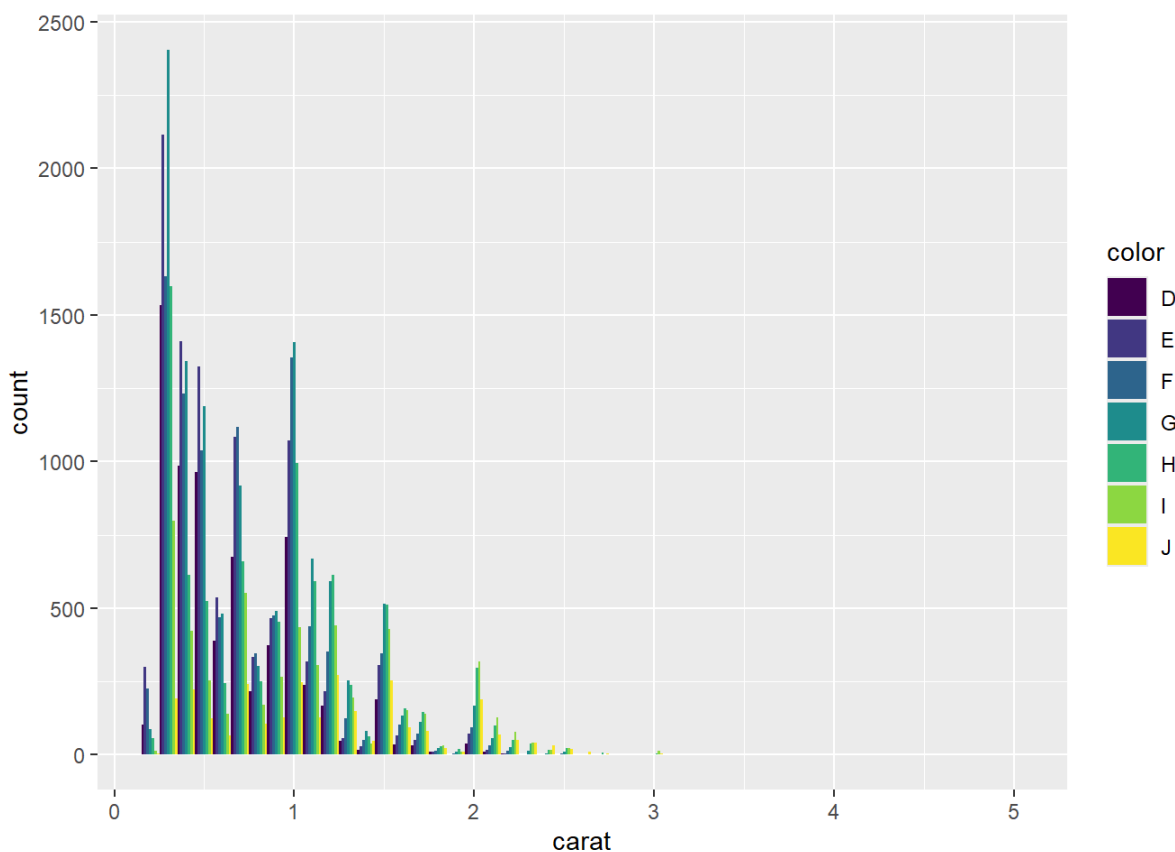
```
# Change the position adjustment to dodge
```

```
s$layers[[1]]$position <- position_dodge()
```

```
t$layers[[1]]$position <- position_dodge()
```

```
# Recall s with Updated Position:
```

```
s
```



```
# Recall s with Updated Position:
```

```
t
```

